

Sistema de Asistencias con Códigos QR

Un sistema web completo para el registro de asistencias utilizando códigos QR dinámicos y seguros.

Características

- **Códigos QR Dinámicos:** Generación automática de códigos QR únicos que se actualizan cada minuto
- **Autenticación Segura:** Sistema de login con roles (Admin/Usuario)
- **Panel de Administración:** Gestión completa de usuarios y registro de asistencias
- **Dashboard de Usuario:** Visualización del código QR personal e historial de asistencias
- **Seguridad Avanzada:** Tokens firmados con expiración automática
- **Interfaz Moderna:** UI responsive con Bootstrap 5 y Font Awesome

Requisitos

- Python 3.8+
- pip (gestor de paquetes de Python)

Instalación

1. Clonar o descargar el proyecto

```
cd qr_flask
```

2. Crear entorno virtual (recomendado)

```
python -m venv .envv  
source .envv/bin/activate # En Windows: .envv\Scripts\activate
```

3. Instalar dependencias

```
pip install -r requirements.txt
```

4. Ejecutar la aplicación

```
python start_server.py
```

O usando el archivo run.py tradicional:

```
python run.py
```

Acceso al Sistema

Una vez iniciado el servidor, accede a: **http://localhost:5000**

Usuarios de Prueba

El sistema crea automáticamente dos usuarios de demostración:

- **Administrador**
 - Usuario: **admin**
 - Contraseña: **admin123**
 - Permisos: Gestión completa del sistema
- **Usuario Regular**
 - Usuario: **usuario**
 - Contraseña: **usuario123**
 - Permisos: Ver su código QR y historial

Uso del Sistema

Para Administradores

1. **Iniciar sesión** con credenciales de admin
2. **Panel de Administración:** Gestionar usuarios del sistema
3. **Registrar Asistencia:** Escanear códigos QR de los usuarios
4. **Crear/Editar Usuarios:** Agregar nuevos usuarios al sistema

Para Usuarios

1. **Iniciar sesión** con sus credenciales
2. **Ver Código QR:** Código personal que se actualiza automáticamente
3. **Historial:** Revisar sus registros de asistencia anteriores
4. **Perfil:** Actualizar información personal

Configuración

Variables de Entorno

Puedes personalizar la configuración creando un archivo **.env**:

```
# Configuración del servidor
FLASK_ENV=development
FLASK_RUN_HOST=0.0.0.0
FLASK_RUN_PORT=5000
```

```
# Seguridad
SECRET_KEY=tu_clave_secreta_muy_segura
QR_SECRET_KEY=tu_clave_para_qr_muy_segura
QR_EXPIRATION=60

# Base de datos
DATABASE_URL=sqlite:///app.db
```

Configuraciones Disponibles

- **QR_EXPIRATION**: Tiempo de vida de los códigos QR en segundos (por defecto: 60)
- **SECRET_KEY**: Clave secreta para sesiones de Flask
- **QR_SECRET_KEY**: Clave para firmar tokens de códigos QR
- **DATABASE_URL**: URL de conexión a la base de datos

Estructura del Proyecto

```
qr_flask/
├── app/
│   ├── __init__.py      # Configuración principal de Flask
│   ├── models/          # Modelos de base de datos
│   │   ├── user.py      # Modelo de usuarios
│   │   ├── role.py      # Modelo de roles
│   │   ├── attendance.py # Modelo de asistencias
│   │   └── base.py      # Modelo base
│   ├── routes/          # Rutas de la aplicación
│   │   ├── auth.py      # Autenticación
│   │   ├── admin.py     # Panel de administración
│   │   └── user.py       # Dashboard de usuario
│   ├── services/         # Lógica de negocio
│   │   ├── user_service.py # Gestión de usuarios
│   │   ├── qr_service.py  # Generación y validación QR
│   │   └── attendance_service.py # Gestión de asistencias
│   ├── templates/       # Plantillas HTML
│   └── utils/            # Utilidades
│       └── qr_generator.py # Generación de códigos QR
├── config.py            # Configuraciones
├── run.py               # Ejecutor tradicional
├── start_server.py      # Ejecutor mejorado
└── requirements.txt     # Dependencias
```

Seguridad

- **Códigos QR Firmados**: Cada QR contiene un token HMAC firmado
- **Expiración Automática**: Los códigos expiran automáticamente
- **Unicidad Diaria**: Un usuario solo puede registrar asistencia una vez por día
- **Autenticación por Roles**: Control de acceso basado en roles

- **Protección CSRF:** Protección contra ataques de falsificación

Solución de Problemas

Error: "No module named 'app'"

```
# Asegúrate de estar en el directorio correcto
cd qr_flask
python start_server.py
```

Error: Puerto ocupado

```
# Cambiar puerto en el archivo .env o usar:
FLASK_RUN_PORT=8000 python start_server.py
```

Base de datos corrupta

```
# Eliminar base de datos y reiniciar
rm instance/app.db
python start_server.py
```

Desarrollo

Agregar Nuevas Funcionalidades

1. **Modelos:** Crear en `app/models/`
2. **Servicios:** Lógica en `app/services/`
3. **Rutas:** Endpoints en `app/routes/`
4. **Templates:** HTML en `app/templates/`

Base de Datos

El sistema usa SQLite por defecto. Para ver la base de datos:

```
sqlite3 instance/app.db
.tables
.schema users
```

Infraestructura IaC con Terraform (AWS Free Tier)

Se agregó una plantilla Terraform en `infra/terraform/` para crear una instancia EC2 en AWS con tipo `t2.micro` y desplegar el proyecto automáticamente.

Recursos que crea

- VPC, subred pública, Internet Gateway y tabla de rutas
- Security Group con puertos:
 - 22 (SSH, restringido por `ssh_cidr`)
 - 5000 (app Flask, configurable)
- Instancia EC2 Amazon Linux 2023 (`t2.micro` por defecto)
- Descarga automática del repo desde `repo_url`
- Instalación de dependencias con `pip`
- Arranque como servicio `systemd` con `gunicorn`

1. Prerrequisitos

- Tener Terraform instalado
- Tener AWS CLI configurado (`aws configure`)
- Tener creado un Key Pair EC2 en tu cuenta (nombre de llave)
- Tener acceso al repositorio Git que vas a desplegar

2. Configurar variables

```
cd infra/terraform
cp terraform.tfvars.example terraform.tfvars
```

Editar `terraform.tfvars` y ajustar al menos:

- `ssh_key_name`
- `ssh_cidr` (recomendado `TU_IP_PUBLICA/32`)
- `repo_url`
- `repo_branch`
- `project_name` si quieres cambiar el nombre del servicio

3. Desplegar

```
terraform init
terraform plan -out tfplan
terraform apply tfplan
```

4. Obtener IP y conectarte

```
terraform output
```

Usa la salida `ssh_command` para conectarte.

5. La app arranca sola

Terraform deja un servicio `systemd` creado y activo. Si necesitas revisar el estado:

```
sudo systemctl status qr-flask  
sudo journalctl -u qr-flask -f
```

6. Eliminar infraestructura (evitar costos)

```
terraform destroy
```

Soporte

Para reportar problemas o sugerir mejoras:

1. Verificar los logs del servidor
2. Revisar la documentación
3. Comprobar configuración de variables de entorno

Licencia

Este proyecto es para fines educativos y de demostración.

¡Sistema listo para usar! 🎉

Inicia el servidor y accede a `http://localhost:5000` para comenzar.